LAPQ lib

Generated from `docs/lapq-lib.ipynb`

Abstract

This report documents lapq, an experimental C99 skip-list priority queue with learning-augmented insertion hints, Python bindings, graph-based dataset generation, and a roadmap toward machine-learning predictors.

Contents

lapq: Learning-Augmented Priority Queue Experiments

This notebook is the living project report for `lapq`. It describes the design of the C library, the Python experimental layer, the current relationship with the reference paper, and the roadmap toward machine-learning predictors. The report is intentionally written as a technical narrative rather than as API documentation: implementation details are included only when they are needed to explain a design decision, an experimental result, or a limitation of the current prototype.

1 Introduction and Motivation

1.1 Project Context

The goal of this project is to study how predictions can be integrated into a priority queue without weakening correctness. The repository therefore has two complementary parts. The first part is a compact C99 library implementing a priority queue based on skip lists. The second part is a Python experimental layer used to load graph instances, generate datasets, replay priority-queue operations, and eventually train predictors.

1.2 Correctness Before Prediction

The central design principle is that predictions are never trusted as proof. A prediction may suggest where a new key should be inserted, but the C data structure must still validate and correct that suggestion using the ordinary comparator. In other words, predictions are treated as advice, while clean comparisons remain the source of correctness. This separation is important both theoretically and architecturally: it allows the Python layer to experiment with arbitrary prediction mechanisms while the C core remains independent from machine-learning code.

1.3 Why Skip Lists

The project originally started from the broader idea of implementing advanced heap-like structures. However, the learning-augmented priority queue literature suggests that heaps are not the most natural target for pointer predictions: the structure does not expose an insertion position in the same way a sorted linked structure does. Skip lists are a better experimental substrate because they combine ordered structure, pointer navigation, and expected logarithmic behavior. This makes them a reasonable bridge between a practical C implementation and the pointer-prediction model discussed in the reference paper.

1.4 Scope of the Current Prototype

At this stage the project should be read as an experimental systems project rather than as a complete implementation of every theoretical component from the paper. The current objective is to build a clean, testable, and measurable foundation, then use experiments to decide which algorithmic and machine-learning extensions are justified.

2 Theoretical Background

2.1 Priority Queues and Heaps

A standard priority queue supports insertion and minimum extraction, and is often implemented with a binary heap. Heaps are excellent general-purpose priority queues, but their operations are organized around an implicit array tree. This is convenient for speed and memory locality, but less convenient when a prediction is supposed to identify a location near the correct insertion point.

2.2 Skip Lists

Skip lists provide a different representation. They maintain elements in sorted order at the bottom level and add randomly sampled higher-level forward pointers that accelerate search. In an ordinary skip-list insertion, the algorithm searches from the head at the top level, descends through the levels, and eventually links the new node at the appropriate position. If a prediction can point close to the correct predecessor, it is natural to ask whether the search can be shortened.

2.3 Learning-Augmented Algorithms

Learning-augmented algorithms study exactly this kind of question: how can an algorithm exploit predictions when they are accurate, while remaining correct and robust when they are inaccurate? In this project, the relevant prediction is not a predicted key value but a predicted structural position, such as a predecessor pointer or an approximate rank.

2.4 Clean and Dirty Comparisons

The reference LAPQ model [BC24] distinguishes between clean comparisons, dirty comparisons, and predictions. The current implementation only has clean comparisons: every comparison that affects correctness is performed by the user-provided comparator in the C core. Dirty comparisons and the full theoretical rank-prediction machinery are left as future work. This means that the project does not yet claim the full theoretical guarantees of the paper. Instead, it implements a practical, measurement-oriented subset focused on predecessor hints and clean-comparison reduction.

3 C Library Architecture

3.1 Public API and Ownership Model

The C library is organized around an opaque `struct lapq` exposed through `include/lapq/lapq.h`. The public API deliberately hides the representation of the queue. This keeps callers independent from the internal skip-list layout and leaves room for future implementation changes without breaking user code.

A key architectural decision is that the queue stores caller-owned `void *` items. The C core does not know what an item means and does not own the item memory. Ordering is entirely determined by the comparator supplied by the caller. This makes the library generic and keeps the learning-augmented layer orthogonal to application data.

3.2 Internal Modules

Internally, the implementation is split into small modules with distinct responsibilities. `src/lapq.c` acts as the public facade: it owns queue creation, destruction, insertion, extraction, and API-level validation. `src/skiplist.c` implements the ordered skip-list structure, including ordinary search, hinted search, insertion, unlinking, and invariant checks. `src/handles.c` manages generational handles, which are used to validate predecessor hints and to support operations such as removal and `decrease_key`. `src/stats.c` collects instrumentation counters, including clean comparisons and traversal steps.

3.3 Handles and Invariants

Handles are central to the design. A predecessor hint is only useful if the C core can check that it refers to a live node in the same queue. The handle table uses generations to detect stale handles, and invalid hints are safely ignored. This is important for robustness: a bad hint may cost extra comparisons, but it must never corrupt the queue.

3.4 Instrumentation

The library exposes optional instrumentation counters. These counters are not required for correctness, but they make the experimental layer possible. In particular, clean-comparison counters allow the experiments to observe the effect of hint quality without relying only on wall-clock time.

4 Prediction Hints in the C Core

4.1 Hint Representation

The public hint interface is represented by `struct lapq_hint`. The current implementation supports three modes: no hint, predecessor hint, and rank hint. The predecessor-hint path is the main learning-augmented mechanism currently used by the experiments.

4.2 Predecessor Hints

When no hint is provided, insertion behaves like a standard skip-list insertion: the search starts from the head and proceeds top-down. When a predecessor hint is provided, the C core first validates the handle. If the handle is invalid, stale, or associated with another queue, the insertion falls back to ordinary search. If the handle is valid, the search starts near the predicted node and corrects the prediction using clean comparisons.

4.3 Hinted Search Phases

The hinted search is split into explicit phases. Backward correction handles the case where the predicted predecessor is too far to the right. Bottom-up expansion uses skip-list levels to move outward from the predicted position. Top-down refinement then completes the insertion search. These phases are measured separately by the instrumentation counters, which makes it possible to understand not just how many comparisons were performed, but also where the search spent its work.

4.4 Rank Hints

Rank hints are currently exposed as an experimental convenience. They are useful for testing and for comparing against rank-like predictions, but the current implementation obtains a starting point by walking level 0. This is not the asymptotically strong rank-prediction structure described in the paper. For this reason, rank hints are treated as part of the experimental API rather than as a final algorithmic claim.

5 Difference from the Paper

5.1 Implemented Subset

The implementation is inspired by the pointer-prediction search model, but it is not a full implementation of the theoretical data structure in the reference paper. This distinction is important for the report.

What is implemented is a practical skip-list priority queue that can accept predecessor hints, validate them through handles, correct them with clean comparisons, and expose detailed instrumentation. The hinted search is algorithmically closer to the pointer-prediction idea than a naive linear correction: it uses skip-list levels to expand and refine the search from the predicted node.

5.2 Missing Theoretical Components

What is not implemented yet includes dirty comparisons, the complete theoretical rank-prediction framework, and auxiliary indexed structures such as a vEB-like mechanism for strong rank hints. The repository also currently has one concrete backend rather than a family of interchangeable priority-queue backends.

5.3 Current Claims and Non-Claims

The current claims are therefore empirical and architectural. The project claims that the C core is correct under arbitrary hints, that good predecessor hints can reduce clean comparisons, and that the Python layer can generate and replay realistic insertion traces from graph workloads. It does not yet claim the full theoretical bounds of the paper.

6 Testing Strategy

The testing strategy follows the separation between the C core and the Python experimental layer. The C tests focus on correctness of the data structure: insertion and extraction order, duplicate keys, handle validity, `decrease_key`, arbitrary removal, invalid hints, stale handles, hinted search phases, and randomized oracle checks. Sanitizer builds are part of `make check`, so memory safety regressions are caught early.

The Python tests focus on the binding and experiment pipeline. They check the CPython `PriorityQueue` wrapper, handle-returning insertions, predecessor and rank hint calls, graph loading, Dijkstra baselines, dataset generation, CSV analysis, and replay. The replay tests are especially useful because they verify that precomputed hint plans produce consistent checksums and expected comparison counters.

This split keeps the repository maintainable. C tests answer whether the priority queue is correct. Python tests answer whether the experimental machinery produces reproducible, meaningful inputs and summaries.

7 C Benchmarks

The C benchmark is the lowest-overhead measurement of the priority queue itself. It compares ordinary insertion and extraction against several hint scenarios: perfect predecessor hints, noisy predecessor hints, intentionally bad hints, and a `decrease_key` workload.

The benchmark reports wall-clock time, clean comparisons, level-0 traversal steps, express-level traversal steps, phase-specific traversal counters, hint counts, invalid-hint counts, average hint error, maximum hint error, and a checksum. The checksum is not a performance metric; it is a simple guard that ensures each benchmark scenario preserves the same logical output.

The C benchmark is useful because it removes Python overhead from the measurement. However, it is synthetic. The Python replay benchmark complements it by using insertion streams derived from Dijkstra runs on real road graphs.

8 Python Binding

The Python package exposes the C core through a CPython extension. The main public class is `PriorityQueue`; opaque handles are represented by `Handle`. The binding exposes insertion, extraction, peeking, removal, invariant checks, statistics, and insertion with predecessor or rank hints.

The binding is intentionally thin. It does not compute predictions and it does not encode machine-learning logic. A Python experiment can compute a hint in any way it wants, but the C extension only receives the resulting handle or rank. This keeps the boundary between prediction and data-structure correctness explicit.

Python-facing `decrease_key` is not exposed yet. The C core supports `lapq_decrease_key`, but the Python wrapper needs a clear policy for mutating priorities associated with Python-owned payloads before exposing that operation safely. For the current Dijkstra experiments, the implementation uses the standard lazy-duplicate approach, which is also how Python `heapq` is commonly used.

9 Graph Experiment Layer

9.1 CSR Graph Representation

The graph layer loads DIMACS shortest-path graphs into a compact CSR representation. It provides Dijkstra baselines with Python `heapq`, Dijkstra with `lapq`, and Dijkstra with synthetic LAPQ hints. Dataset utilities emit CSV rows for priority-queue insertion events, including Python-computed oracle predecessor information.

9.2 Dataset Generation Workflow

The current dataset workflow is intentionally simple and reproducible. A road graph is loaded once, a fixed set of pseudo-random sources is selected from a seed, Dijkstra is run once per source, and at most a fixed number of queue-insertion events is emitted per run. The current New York dataset snapshot was generated from `USA-road-d.NY.gr` with eight sources, seed `123`, and at most 50,000 events per source.

9.3 Queue-Insertion Event Schema

The generated queue-insertion CSV stores only data that is useful for the current experiments. The fields are grouped by role rather than by implementation detail:

```
Identification:       run, step, source, node, target
Queue context:        distance, edge_weight, queue_size_before
Oracle predecessor:   predecessor_rank, predecessor_distance,
                      predecessor_node, predecessor_sequence
```

The identification fields locate the event inside a Dijkstra run. The queue-context fields describe the insertion being replayed. The oracle-predecessor fields record the predecessor that was computed during dataset generation and are used by replay experiments and future supervised-learning targets.

The road graph files under `graphs/dimacs/` are local data and are intentionally kept out of Git.

10 Experimental Methodology and Metrics

10.1 Primary Metric: Clean Comparisons

The main algorithmic metric is the number of **clean comparisons** performed by the C priority queue. Wall-clock time is still recorded, but it is treated as an engineering metric rather than the primary scientific signal.

This choice is important because the Python experiments include costs that are not part of the C data structure itself: CPython call overhead, Python handle management, synthetic hint generation, and auxiliary oracle structures. These costs can hide the algorithmic effect of predictions. Clean comparisons instead directly measure how much correctness-critical work remains after a hint is supplied and corrected.

10.2 Experiment Types

The current Python pipeline uses three complementary experiment types:

1. **Dijkstra backend comparison.** Runs shortest paths with `heapq` and with `lapq` without hints.
2. **Dijkstra synthetic hints.** Runs Dijkstra with LAPQ and synthetic hint policies while preserving the full Dijkstra workflow.
3. **Replay benchmark.** Replays a saved stream of priority-queue insertions after precomputing hint plans. This removes most graph/oracle overhead and focuses on how the queue behaves when hints are already available.

10.3 CSV Reporting

CSV reports are organized so that comparison counts, comparison ratios, and comparison reductions appear before timing columns. This convention is intentionally used throughout the Python experiment layer to keep the analysis focused on algorithmic work.

11 Current Road-Graph Results

11.1 Dataset Snapshot

The current road-graph snapshot uses the DIMACS New York graph with eight pseudo-random sources. The generated queue-insertion dataset contains 400,000 events, split evenly across eight Dijkstra runs.

The dataset-level summary shows that each run contributes 50,000 events. The average active queue size is in the hundreds, which makes predecessor prediction nontrivial while keeping replay experiments manageable.

11.2 Replay Results

The most important current result comes from replaying the saved insertion stream and measuring clean comparisons relative to the LAPQ baseline:

scenario	rows	clean comparisons	comparison ratio	comparison reduction	average error	max error
baseline	400,000	12,383,515	1.000	0.00%	0.000	0
perfect	400,000	2,391,779	0.193	80.69%	0.000	0
noisy	400,000	8,546,554	0.690	30.98%	63.982	64
bad_left	400,000	20,922,309	1.690	-68.95%	128,375.903	399,985

This validates the intended learning-augmented behavior: accurate predecessor hints substantially reduce clean comparisons, moderately noisy hints can still help, and adversarially bad hints increase work while preserving correctness.

11.3 Dijkstra-Level Synthetic Hints

The Dijkstra-level synthetic hint experiment tells a complementary story. Perfect and rank hints reduce clean comparisons by 63.68% relative to the no-hint LAPQ baseline on the single-source NY run, while the current noisy and bad-left policies increase comparisons in that full Dijkstra setting. These timing numbers should not be over-interpreted, because the Python oracle and hint-generation path are still part of the measured end-to-end runtime.

12 Machine Learning Roadmap

12.1 Prediction Target

Real ML models are intentionally out of scope until the dataset format, replay workflow, and baseline measurements are stable. The current repository now has the required scaffolding: DIMACS loading, CSR graphs, multi-source dataset generation, oracle predecessor extraction, comparison-first analysis, and replay.

The first supervised target should be a predecessor-oriented quantity rather than a raw C handle. Candidate targets include:

- predecessor rank/order in the active priority queue;
- an approximate predecessor bucket;
- a hybrid policy that decides when to provide no hint.

12.2 Feature Extraction

Candidate features may come from Dijkstra state, graph topology, current distance values, queue size, edge weights, source/target identifiers, and normalized event position.

Placeholder. This subsection should define the exact feature schema before training models.

12.3 First Models

The first models should be deliberately simple: heuristic baselines, linear or tree-based regressors/classifiers, and only then heavier ML methods if the simpler models justify them.

Placeholder. This subsection should describe train/test splits, validation metrics, and the first non-ML baselines.

13 Algorithmic Roadmap

The C core is complete for the current v0.1 experimental scope, but it is not the end of the algorithmic story. Future work should be guided by measurements rather than by adding complexity prematurely.

The most important missing theoretical component is dirty comparisons. Adding them would require a careful API and a clear distinction between cheap predicted comparisons and correctness-critical clean comparisons. Another major direction is stronger rank hints. The current rank-hint path is intentionally simple and walks level 0 to locate the predicted rank. A serious rank-prediction implementation would need an auxiliary indexed structure or a different skip-list augmentation.

A backend abstraction may become useful later, but it is not necessary yet. Adding a vtable or multiple backends before the experiments justify it would make the C core harder to reason about. For now, the cleanest path is to keep one well-tested skip-list backend and use Python experiments to identify the next algorithmic bottleneck.

14 Packaging and Reproducibility

The project uses `setuptools` and a CPython extension. GitHub Actions runs tests and builds wheels for Linux, macOS, and Windows. Doxygen API documentation can be generated locally with `make api-docs`; the generated HTML is ignored by Git.

Release artifacts are built from Git tags and attached to GitHub Releases.

15 Current Status

Completed components include the C skip-list priority queue, handles, predecessor hints, experimental rank hints, instrumentation, C benchmarks, Python bindings, DIMACS/CSR graph loading, Dijkstra baselines, synthetic hint scenarios, multi-source dataset CSV generation, comparison-first analysis, replay benchmarks, packaging, CI, and Doxygen infrastructure.

Current limitations include the absence of dirty comparisons, strong rank-aware structures, Python-facing `decrease_key`, and real machine-learning models.

The current empirical story is strong enough to guide the next phase: on the NY replay dataset, perfect predecessor hints reduce clean comparisons by 80.69%, noisy hints with bounded error still reduce them by 30.98%, and bad hints increase work without compromising correctness.

16 Conclusion

The repository currently provides a stable experimental foundation: a compact C core with clean correctness semantics and a Python layer designed to generate, replay, and evaluate predictions. The experiments now use clean comparisons as the primary algorithmic metric, which makes the effect of prediction quality visible despite Python-level overheads.

The next research step is controlled feature extraction and simple supervised prediction. The goal is not to introduce ML for its own sake, but to test whether learned predecessor hints can reproduce some of the comparison savings observed with oracle and synthetic hints.

Bibliography

[BC24] Ziyad Benomar and Christian Coester. Learning-augmented priority queues. In *Advances in Neural Information Processing Systems 38*, 2024. Local copy: docs/papers/learning_augmented_priority_queues.pdf.